SAMPLE ASSIST · ENGINEERING

Intern Project Brief

Consumables Ordering & Marketplace for Collection Manager

FOR	DURATION	DATE
Software engineering intern	12 weeks · fully self-paced over 3 months	15 Jul 2026

1 Welcome and context

Welcome to the team. This document describes a project you will own over the next three months, entirely at your own pace. Read it in full before you write any code. This brief is the source of truth for the project — there is no starter repository. You will set up the codebase yourself from scratch: folder layout, environment variables, scripts, and coding conventions are all yours to decide and to document as you go.

Sample Assist builds software for workplace drug and alcohol testing. Our product Collection Manager (CM) is the control centre our collection agencies use to manage staff, scheduling, multi-site operations, compliance records, and inventory of testing consumables — things like urine and saliva test cups, cassettes, breathalysers, chain-of-custody pads, and absorbent underpads.

Today a collection manager can track inventory in CM but cannot reorder stock from inside the product. This project closes that gap and adds a second surface: a public storefront where anyone — not just CM customers — can buy the same consumables, and where approved sellers can list and sell their own products. Think of a focused, drug-and-alcohol-testing version of the seller/buyer model used by Amazon Marketplace or Shopee.

2 What we are building

There are two connected surfaces backed by one shared system.

A. Consumables ordering inside Collection Manager (for CM users)

- A collection manager browses a catalogue of consumables and places an order without leaving CM.
- Each completed order produces an electronic invoice (a downloadable PDF tax invoice, GST-inclusive, matching Australian norms).
- Placing and receiving an order updates the manager's inventory automatically, so stock counts stay accurate.

B. Public storefront / marketplace (for everyone)

- A buyer-facing e-commerce site: browse, search, filter, add to cart, check out, and download invoices — no CM account required.
- A seller side where a business can register, get verified, create and manage product listings, and issue invoices for their sales.
- Seller verification: we must confirm a seller is a legitimate registered business before their listings go live.

C. One shared backend (important)

Both surfaces are powered by the same backend and the same database. If a person is both a CM user and a storefront buyer, they see the same account, the same order history, and the same inventory across both. **Do not build two separate systems that sync — build one system with two front-end entry points.** This is the single most important architectural decision in the project.

3 Scope: phased MVP, then stretch

Three months for one developer is not enough time to rebuild Amazon, so the scope is deliberately layered. Phase 1 is the committed MVP — this is what "done" means for the project. Phases 2 and 3 are stretch goals: attempt them only once Phase 1 is complete, tested, and reviewed. It is far better to deliver a solid, well-tested MVP than a broad set of half-finished features.

PHASE 1 — MVP (THE COMMITTED DELIVERABLE, ROUGHLY THE FIRST HALF OF YOUR TIME)

Area	What must work
Accounts & roles	Registration and login. Roles: buyer, seller, collection-manager, and admin. A single account can hold more than one role.
Catalogue & product	Products with name, description, images, category, price (GST-inclusive), and stock quantity. Seller-owned listings.
Browse & search (buyer)	Product listing page with search by keyword and filter by category and price range. Product detail page.
Cart & checkout	Add to cart, edit quantities, checkout. Payment is mock/sandbox only (see Section 4) — no real money moves.
Orders	Order creation, order status (e.g. placed → confirmed → fulfilled), and an order-history view for buyers and sellers.
Electronic invoice	Each order generates a downloadable PDF tax invoice: seller & buyer details, line items, GST, totals, invoice number, date.
Seller onboarding	Seller sign-up, submit business details (e.g. ABN, business name, contact), create/edit/deactivate their own listings.
Seller verification	Admin can review a pending seller and approve or reject. Listings from unverified sellers are not visible to buyers.
CM ordering	Inside Collection Manager, a CM user can browse the catalogue and place an order using the same backend.
Inventory update	Placing/receiving an order adjusts the CM user's inventory counts automatically.
Shared data	A user who is both a CM user and a buyer sees one consistent account, order history, and inventory across both surfaces.

PHASE 2 — MARKETPLACE DEPTH (STRETCH, IF MVP IS DONE AND SOLID)

- Richer buyer filters and sorting: brand, drug panel / test type, pack size, availability, rating; sort by price, newest, popularity.
- Seller dashboard with basic sales analytics (orders, revenue, best-sellers) and low-stock alerts.
- Seller storefront pages, product ratings/reviews, and buyer order tracking with status notifications.
- Automated business-number (ABN) format validation and, if a free/public lookup is available, a verification lookup to assist admin review.
- Inventory features: reorder thresholds, purchase-order history, and a stock movement log.

PHASE 3 — NICE TO HAVE (STRETCH)

- Redis caching (Docker) for catalogue and search results to speed up browse pages — this is the explicit bonus goal.
- Real payment integration in test mode (e.g. Stripe test keys) layered on top of the mock checkout.
- Email notifications (order confirmation, invoice delivery) via a local mail-catcher such as Ethereal.
- Discount codes / bulk & trade pricing tiers, and a basic admin reporting dashboard.

Explicitly out of scope: real card processing or moving real money, shipping-carrier integrations, mobile apps, multi-currency, and production deployment. Everything runs locally.

4 Payments and electronic invoices

Use a mock / sandbox checkout only. The checkout flow should look and behave realistically — collect the order, confirm it, and mark it paid — but no real payment gateway is connected and no real money moves. This keeps the project safe to run locally and avoids handling anyone's card data.

Regardless of payment being mocked, every completed order must generate a proper electronic tax invoice as a downloadable PDF. Model it on a standard Australian tax invoice: a unique invoice number, issue date, seller identity (business name and ABN), buyer details, itemised lines with quantities and unit prices, GST shown, and the total. Prices are GST-inclusive. Store invoices so they can be re-downloaded later from order history.

If you reach Phase 3, you may add Stripe in test mode on top of the mock flow — but never with live keys, and never committed to version control.

5 Tech stack and environment

You will build a full-stack monorepo from scratch. Use the stack below — do not swap frameworks. You choose the exact versions: pin them and document your setup so the project runs from a clean checkout.

Layer	Technology	Notes
Frontend	React + Vite	Single-page app. Shares components/patterns with the CM front end where sensible.
Backend	NestJS	REST API. Modular structure (auth, products, orders, invoices, inventory, sellers).
ORM	TypeORM	Entities and migrations. Do not hand-edit the database; use migrations.
Database	PostgreSQL (Docker)	Runs as a local Docker container.
Cache (bonus)	Redis (Docker)	Optional, Phase 3. Local Docker container for caching.
Everything	Local only	No cloud deployment. Use docker-compose.

Set up and document these yourself: environment variables, how to start the databases, seed data, migration commands, linting, and your git workflow. Record your decisions in the README so anyone can run the project from a clean checkout.

6 Definition of done and pacing

A feature is "done" only when all of these are true:

- It meets the requirement described here (and any clarifications you record as you go).
- It has tests: at minimum, backend unit/integration tests for the API logic, and the feature works end-to-end through the UI.
- It passes linting and the project's type checks with no new errors or warnings.
- Database changes are made through TypeORM migrations, not manual edits, and the app runs cleanly from a clean checkout plus your documented setup.
- It lands on `main` through a pull request you open and self-review — keep history clean and never push broken code to main.
- Anything non-obvious is documented in the repo (README or code comments).

Pacing yourself

This project is fully self-paced over the three months — there is no daily check-in, standup, or scheduled review on our side. You own your plan and your time. To stay on track, we suggest a rhythm you set and run for yourself:

Rhythm	Suggested self-practice
Weekly	Set your own goals for the week and track them in the repo (a simple TODO or issues board). Note what you finished, what's next, and any blockers.
Per feature	Open a pull request, keep it small and focused, and self-review it against the definition of done before merging.
End of each phase	Check your work against Section 3. Only move to the next phase once the current one is complete, tested, and solid.
Month 3	Deliver a final demo, handover notes, and a short write-up of what's built, what isn't, and known issues.

If you get stuck, timebox it — try an alternative approach, then work around the blocker and record it in your write-up. Making a reasonable, documented decision and moving on is exactly what we want from a self-directed project.

7 Working guardrails (please read carefully)

- **Use only the Claude subscription we provide.** Do not use your personal AI subscriptions or accounts for any part of this project. All AI-assisted work goes through the company-provided access.
- **This brief is the source of truth.** You own the technical setup and its documentation — keep the README current so your decisions are traceable.
- **Everything runs locally.** No deploying to the cloud, no pushing data to external services. Databases run in Docker on your machine.
- **Never commit secrets.** No API keys, passwords, or .env files in git. Use an environment-variable pattern; keep real values out of version control.
- **No real payments, no real personal data.** Checkout is mocked. Use fake/seed data for testing — never real customer or health information.
- **Mind the sensitivity of the domain.** This is drug-and-alcohol-testing software. Handle any test-related data carefully and follow basic security hygiene: validate input, hash passwords, guard every API route by role, and never expose one user's data to another.
- **Own and document your decisions.** Adding a major dependency, changing the data model significantly, or deviating from this brief — record the decision and your reasoning in the repo.
- **Small, frequent commits and PRs.** Easier to review, easier to unwind if something breaks.
- **Code quality.** No code should be generated by AI without revision. Avoid code smell as much as possible.

8 Getting started

1. Set up the monorepo from scratch and get it running locally end-to-end (frontend, backend, Postgres in Docker). Confirm you can log in and load a page.
2. Read this brief again. Write down your assumptions and open questions, and record them in the repo so your decisions stay traceable.